电 子 科 技 大 学

实 验 报 告

学生姓名：吕乐彬　　学号：2023080904007　　指导教师：李忻洋

一、实验项目名称：信息对抗综合设计实验II——病毒初见

二、实验目的：

本实验旨在通过对可执行程序机器码的分析与修改，理解恶意代码模型的基本执行机制，掌握程序执行流从正常函数转入“病毒函数”并在执行结束后返回原流程的具体实现方法。通过本实验，进一步熟悉 Visual Studio 反汇编窗口、x32dbg/x64dbg 调试工具以及十六进制文件编辑工具的基本使用，提升二进制代码分析、调试与修改能力。

三、实验原理：

本实验的核心原理是通过修改程序入口处的机器码，改变原有控制流，从而重现恶意代码“窃取执行权—执行自身代码—归还执行权”的基本过程。实验中以 `normalfunc()` 作为正常执行流程，以 `virusfunc()` 作为插入的“病毒代码”。首先在 `normalfunc()` 入口写入一条 JMP 指令，使程序执行到该处时不再继续原有流程，而是跳转到 `virusfunc()` 执行指定代码；随后在 `virusfunc()` 的空闲区域补写被覆盖的原始指令，并在其后加入回跳指令，使程序重新返回 `normalfunc()` 中被覆盖区域之后的位置继续执行。由此形成完整的控制流劫持与恢复闭环。

本实验的关键不在于单纯插入一条跳转指令，而在于保证原程序语义不被破坏。由于首跳 JMP 会覆盖 `normalfunc()` 入口处的部分原始机器码，因此必须按照完整指令边界恢复被覆盖的原始指令，否则程序在执行完 `virusfunc()` 后将无法正常返回，原有功能也不能继续完成。这说明恶意代码模型的本质并不是“额外执行一段代码”，而是在不破坏原程序整体功能的前提下，临时接管其执行流程。

实验中使用的 JMP 为近跳转指令，其机器码形式通常为 `E9 xx xx xx xx`，其中后 4

个字节为相对偏移量。偏移量需依据源地址与目标地址计算，并按小端方式写入。因此，无论是在 Visual Studio 中修改内存、在十六进制编辑器中修改可执行文件，还是在 x64dbg 中通过汇编编辑功能完成修改，其本质都是对最终执行机器码的改写。通过这种方式，可以直观理解程序控制流如何被劫持、恢复以及保持连续性，这也是恶意代码分析、Hook 技术和二进制补丁中的基础思想。

四、实验环境（设备、元器件）：

硬件环境：PC 机一台。

软件环境：Windows 操作系统、Visual Studio 开发环境、x32dbg/x64dbg 调试器、ImHex/UE/C32ASI 等十六进制文件编辑工具。

实验对象：使用 Visual Studio 编译生成的 32 位可执行程序。为避免地址随机化对调试结果造成影响，实验前关闭了程序的随机基址选项。

五、实验步骤：

任务1：在内存中（使用VS）重现以上病毒模型的执行过程

1. 编写实验程序并确认基本功能

首先在 Visual Studio 中编写实验程序，设置两个关键函数：`normalfunc()` 和 `virusfunc()`。其中，`normalfunc()` 用于输出 `hello`，表示程序原本的正常执行流程；`virusfunc()` 用于输出本人学号后六位 904007，用于模拟病毒代码执行后的效果。

根据实验要求，`virusfunc()` 最后的 `nop` 数量应为“5+学号末位数”。本人学号为 2023080904007，末位数字为 7，因此在 `virusfunc()` 中共预留了 12 个 `nop` 指令，为后续恢复原始指令和构造回跳逻辑提供空间。

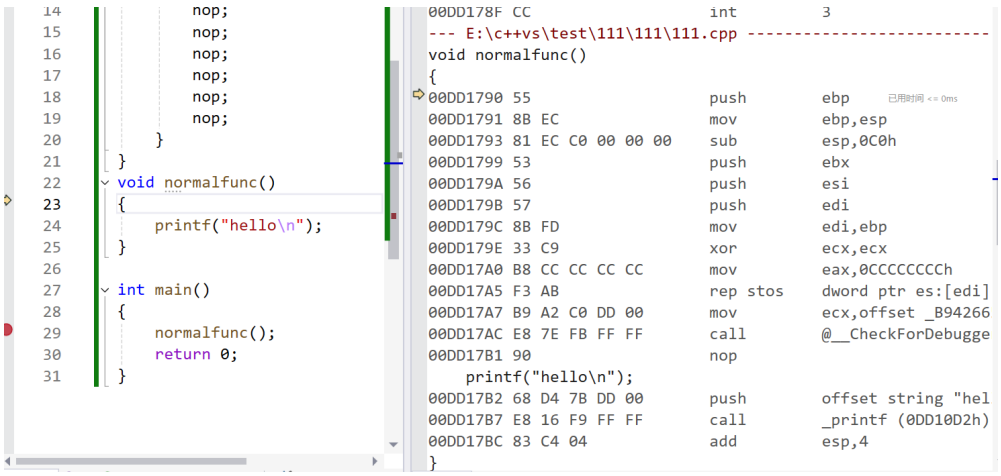

图 1 Visual Studio 中 `normalfunc()` 与主函数的原始代码及初始反汇编位置

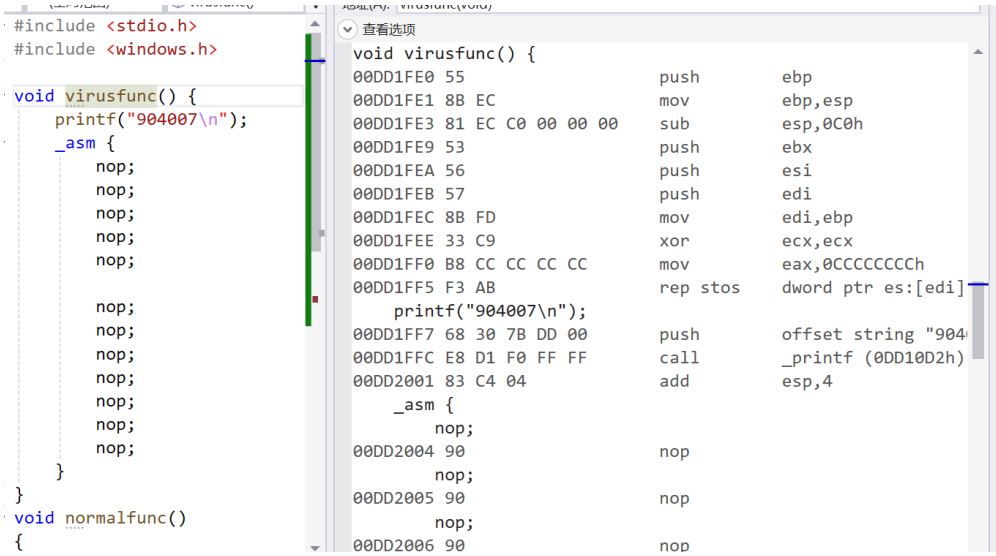

图 2 virusfunc() 中预留的大段 nop 空间以及函数初始反汇编位置

由图1可知，程序原始情况下只会执行 normalfunc()，不会主动执行 virusfunc()。由图2可知，virusfunc() 中已经预留了足够的空闲空间，可用于后续补写被覆盖的原始指令以及构造回跳逻辑。因此，若要让程序先输出 904007，必须通过修改内存中的机器码来改变程序控制流。

2. 在 VS 的反汇编窗口中抓取真实地址

在 Visual Studio 中按 F5 启动调试，并在断点处暂停程序后，按 Alt+8 打开“反汇编窗口”。在该窗口中，需要首先抓取构造首条 JMP 指令所需的两个关键地址。

第一个是源地址，也就是首条 JMP 将要写入的位置。在本实验中，该位置是 normalfunc() 的第一条指令 push ebp 所在地址：

0x00DD1790

第二个是目的地址，也就是程序被强制跳转到的位置。在本实验中，该位置是 virusfunc() 中真正执行打印 904007 的指令地址：

0x00DD1FF7

3. 根据真实地址计算首跳 JMP 的偏移量并拼接机器码

x86 架构下的近跳转指令 JMP 使用的是相对偏移量，而不是直接写入绝对地址。其计算公式为：

$$\text{偏移量} = \text{目的地址} - (\text{源地址} + 5)$$

其中，之所以要加 5，是因为近跳转指令 E9 xx xx xx xx 本身总长度为 5 个字节。当 CPU 取完这条 JMP 指令后，指令指针已经自然移动到了“源地址 + 5”的位置，因此偏移量必须以该地址为基准计算。

将本实验中的真实地址代入，可得：

$$偏移量 = 0x00DD1FF7 - (0x00DD1790 + 5)$$

$$偏移量 = 0x00DD1FF7 - 0x00DD1795$$

$$偏移量 = 0x0862$$

接下来需要将该偏移量写成 4 字节，并按照 Intel/AMD 架构的小端方式存储。首先将 0x0862 补齐为 32 位：

```
00 00 08 62
```

再按字节逆序后得到：

```
62 08 00 00
```

最后，在其前面加上 JMP 指令的操作码 E9，即可得到首跳对应的完整机器码：

```
E9 62 08 00 00
```

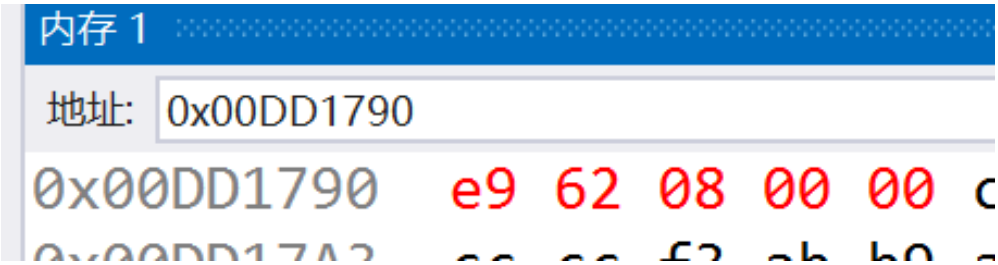

图 3 VS 内存窗口中定位到首跳注入位置附近的机器码

这条机器码的含义是：当程序执行到 normalfunc() 入口地址 0x00DD1790 时，不再继续执行原来的函数序言，而是直接跳转到 virusfunc() 中地址 0x00DD1FF7 处执行打印 904007 的代码，从而实现对正常控制流的劫持。

4. 在 virusfunc() 的空闲区域中恢复原始指令并计算回跳机器码

在本实验中，恢复原始指令的区域从地址 0x00DD2004 开始，因此先将被覆盖的完整函数序言写入该处：

```
55 8B EC 81 EC C0 00 00 00
```

恢复完这 9 个字节后，需要继续在其后添加一条回跳 JMP，使程序重新返回到 normalfunc() 中被覆盖区域之后的位置继续执行。由于原始函数序言共 9 个字节，因此真正应返回的位置为：

```
0x00DD1799
```

而回跳 JMP 的写入地址为：

0x00DD200D

由于 JMP 指令本身长度仍为 5 字节，因此其下一条指令地址为：

0x00DD2012

于是回跳偏移量计算如下：

$$偏移量 = 0x00DD1799 - (0x00DD200D + 5)$$

$$偏移量 = 0x00DD1799 - 0x00DD2012$$

$$偏移量 = 0xFFFFF787$$

按小端方式写入后得到：

87 F7 FF FF

因此，回跳 JMP 的完整机器码为：

E9 87 F7 FF FF

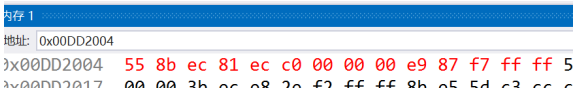

图 4 VS 内存窗口中查看恢复指令与回跳机器码

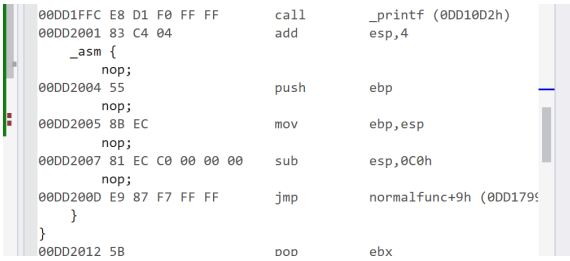

图 5 VS 反汇编窗口中观察恢复代码与回跳逻辑

如图4所示，在 0x00DD2004 开始的位置已经写入恢复的函数序言和回跳机器码；如图5所示，反汇编窗口中可以清楚看到：程序在输出 904007 后，会先执行补写的原始函数序言，再通过 E9 87 F7 FF FF 跳回到 0x00DD1799，从而继续执行 normalfunc() 后续原本应执行的代码。

经过上述操作，程序控制流形成了完整闭环：先由 normalfunc() 跳入 virusfunc()，执行“病毒代码”后，再通过回跳返回 normalfunc() 的后续位置继续执行，从而完成“窃取执行权—归还执行权”的全过程。

6. 运行程序并验证 VS 内存重现结果

完成首跳注入、原始指令恢复和回跳构造后，在 Visual Studio 调试环境中继续运行程序。程序运行结果显示，先输出 904007，随后输出 hello，说明在内存中已经成功重现了恶意代码模型的执行过程。

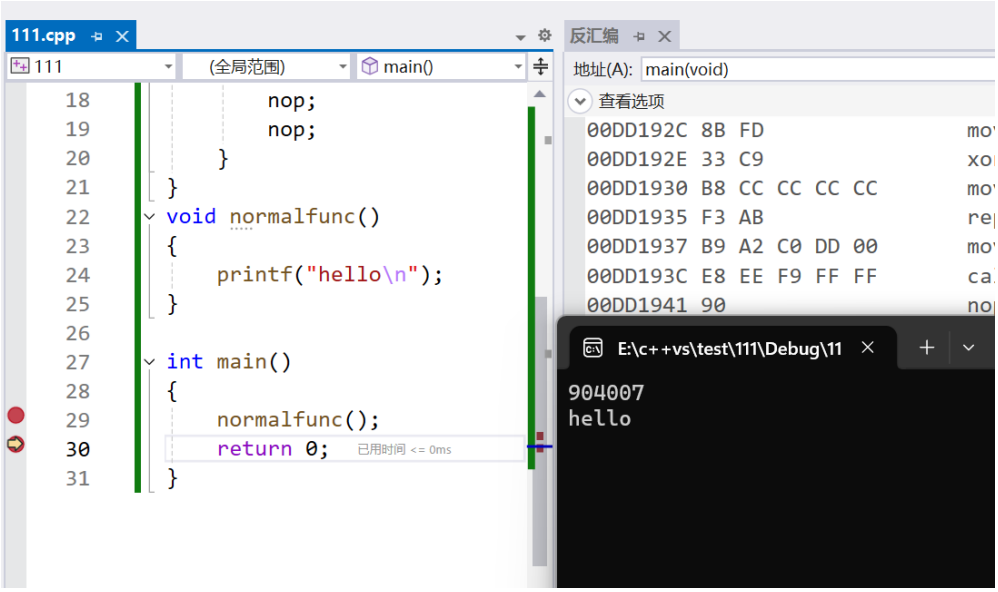

图 6 在 VS 中运行程序，结果显示先输出 904007，再输出 hello

由图6可知，首跳、恢复原始指令以及回跳逻辑均已生效，任务1中“在内存（使用 VS）上重现以上病毒模型的执行过程”已成功完成。

（二）任务1：硬盘（使用 ImHex/UE/C32ASM 等）上重现以上病毒模型的执行过程

1. 打开目标可执行文件并定位 normalfunc() 入口

在完成 VS 内存重现后，进一步使用 ImHex、UE 或 C32ASM 等十六进制编辑工具直接打开由 Visual Studio 编译生成的目标可执行文件。与前一部分不同，此处修改的对象不再是运行中的进程内存，而是硬盘上的 exe 文件本体。

由于文件编辑器中显示的是文件中的原始字节内容，而不是 VS 反汇编窗口中的虚拟地址，因此本实验采用特征码搜索的方式定位 normalfunc() 入口。根据前面在 VS 中的分析，normalfunc() 开头的函数序言字节可作为定位依据，因此在十六进制编辑器中搜索特征码 55 8B EC 81 EC，即可找到目标位置。

图 7 在十六进制编辑工具中按十六进制方式搜索特征码 55 8B EC 81 EC

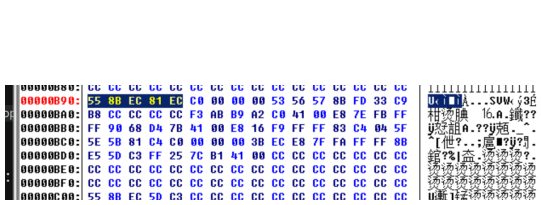

图 8 搜索到 normalfunc() 入口附近的原始机器码

2. 写入首跳机器码

首跳机器码在前面的 VS 内存重现实验中已经根据真实地址计算完成，其结果为：

E9 62 08 00 00

因此，在文件中定位到 normalfunc() 入口后，直接将原始函数序言开头的 5 个字节改写为上述机器码，使程序执行到该函数时优先跳转到 virusfunc() 中输出 904007 的位置。

图 9 在硬盘文件中将 normalfunc() 入口改写为首跳机器码 E9 62 08 00 00

3. 在 virusfunc() 的空闲区域中补写恢复代码与回跳代码

完成首跳注入后，还需在 virusfunc() 预留的空闲区域中补写被覆盖的原始函数序言，并在其后补写回跳指令。前面在 VS 中已完成分析并得到结果：需要恢复的原始字节为

55 8B EC 81 EC C0 00 00 00

回跳机器码为

E9 87 F7 FF FF

因此，在文件中找到 virusfunc() 附近的大段 nop 空间后，先写入恢复代码，再紧接着写入回跳机器码，使程序在执行完 virusfunc() 后能够重新跳回 normalfunc() 的后续位置继续执行。

图 10 文件中 virusfunc() 附近的大段空闲字节区域

图 11 在文件中补写恢复代码与回跳逻辑后的结果

（三）任务2：使用 x64dbg 重现以上恶意代码模型的执行过程（使用 VS 编译生成的可执行程序），并保存可执行文件到硬盘

1. 载入目标程序并定位关键位置

本实验中由 Visual Studio 编译生成的是 32 位可执行程序，因此在 x64dbg 调试套件中实际使用的是 x32dbg。打开调试器并载入目标程序后，在反汇编窗口中分别定位 normalfunc() 与 virusfunc() 的关键位置：一是 normalfunc() 的函数入口，二是 virusfunc() 中输出 904007 的位置，三是 virusfunc() 中用于补写恢复代码和回跳逻辑的空闲区域。

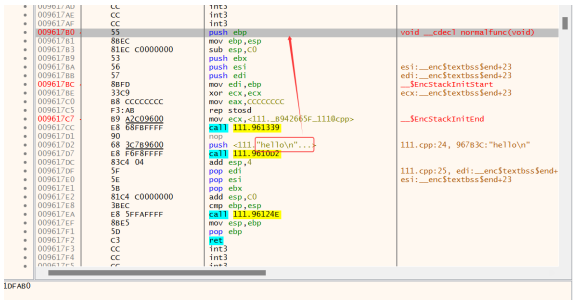

图 12 在 x32dbg 中定位 normalfunc() 及其输出 hello 的相关代码

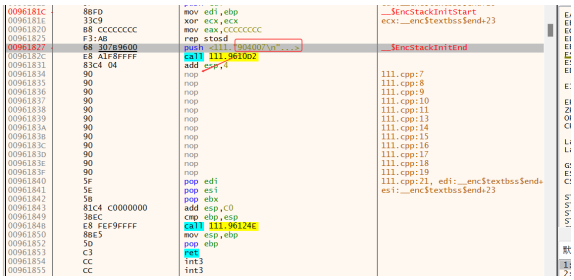

图 13 在 x32dbg 中定位 virusfunc() 中输出 904007 的关键位置

2. 改写 normalfunc() 入口处的原始汇编，构造首跳

在 x32dbg 中，原本 normalfunc() 入口处的汇编代码为函数序言，即：

```
push ebp
mov ebp, esp
sub esp, 0C0h
```

对应的原始机器码为：

```
55 8B EC 81 EC C0 00 00 00
```

实验中将入口开头改写为一条首跳指令，使程序不再直接执行上述函数序言，而是先跳转到 virusfunc() 中打印 904007 的位置。前面在 VS 内存重现实验中已经计算得到首跳机器码为：

```
E9 62 08 00 00
```

因此，本步骤中实际完成的“修改汇编”就是：将 normalfunc() 入口原来的函数序言起始部分改写为一条 jmp 指令，使控制流先跳转到 virusfunc()。

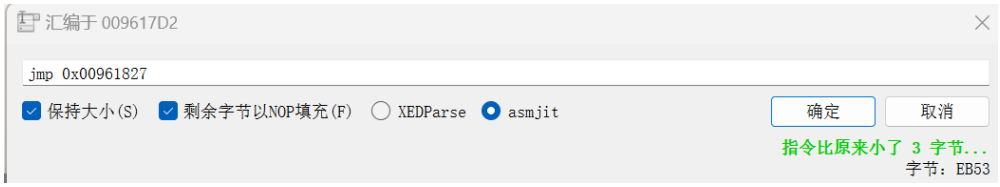

图 14 在 x32dbg 中输入首跳 jmp 指令，改写 normalfunc() 入口

3. 在 virusfunc() 的空闲区域中补写新增汇编代码

完成首跳后，还必须在 virusfunc() 的空闲区域中补写新增汇编代码，以恢复被覆盖的原始函数序言，并在其后添加回跳逻辑。实验中新增写入的汇编代码包括两部分。

第一部分是恢复原始函数序言，即补写以下三条汇编指令：

```
push ebp
mov ebp, esp
sub esp, 0C0h
```

其对应机器码为：

```
55 8B EC 81 EC C0 00 00 00
```

第二部分是在恢复指令之后继续新增一条回跳指令：

```
jmp 0x00DD1799
```

该回跳指令对应的机器码在前面 VS 内存重现实验中已经计算得到，为：

```
E9 87 F7 FF FF
```

因此，在 x32dbg 中本步骤实际“新增”的汇编逻辑可以概括为：

```
push ebp
mov ebp, esp
sub esp, 0C0h
jmp 0x00DD1799
```

它的作用是：程序在 virusfunc() 中完成 904007 的输出后，不是直接结束，而是先补做原本在 normalfunc() 开头应执行的函数序言，再跳回 normalfunc() 中被覆盖区域之后的位置，继续执行原函数剩余逻辑。

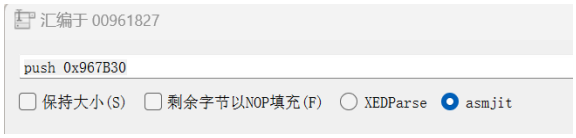

图 15 在 x32dbg 中继续补写恢复代码

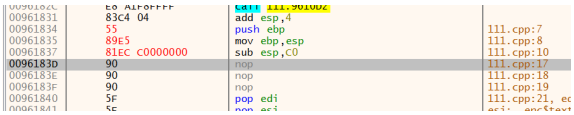

图 16 在 x32dbg 中查看补写完成后的恢复代码与回跳逻辑

4. 保存修改结果并验证最终效果

在 x32dbg 中完成上述修改后，将修改结果保存回可执行文件。重新运行保存后的程序，可以观察到程序先输出 904007，随后输出 hello，说明首跳改写、恢复代码补写以及回跳逻辑都已生效。

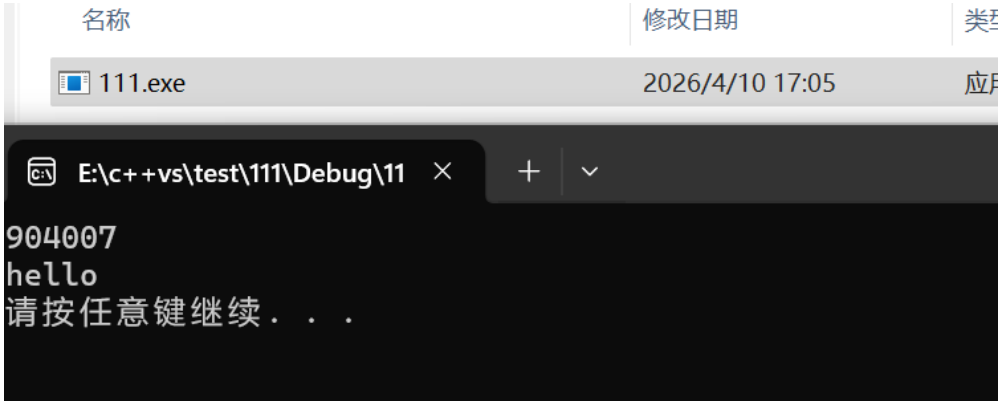

图 17 保存到硬盘后的可执行文件运行结果验证

这表明本部分中完成的关键修改包括：一是将 `normalfunc()` 入口改写为首跳 `jmp`；二是在 `virusfunc()` 空闲区域新增恢复原始函数序言的三条汇编指令；三是在恢复代码后新增一条返回 `normalfunc()` 的回跳指令。至此，任务3“使用 x64dbg 重现以上恶意代码模型的执行过程，并保存可执行文件到硬盘”已经完成。

任务3：拓展内容

本实验中通过首跳与回跳构造了“控制流重定向”模型。该思想虽然常见于恶意代码，但并不只服务于恶意用途。在安全防护、程序调试、外挂对抗、系统监控等场景中，也都存在类似的入口接管与流程恢复机制。因此，本实验的本质不仅是认识病毒模型，更是在理解一种典型的代码控制流劫持与恢复技术。

1. 双重跳转并非只用于恶意代码

双重跳转的核心是：先把程序执行流从原函数入口引走，在完成额外逻辑后，再跳回原函数继续执行。恶意代码可利用这一机制隐藏执行和插入功能；而安全软件、调试器、监控系统也可能利用类似机制对关键函数进行拦截、审计与保护。因此，双重跳转本身是一种中性的技术，其安全属性取决于使用目的。

2. 代码钩子与中间人思想

本实验中的模型实际上已经具有代码钩子（Hook）的基本思想，即在原始调用路径中插入一个“中间层”，使程序先执行额外逻辑，再返回原流程继续运行。这与网络安全中的中间人攻击在结构上是相似的：两者都不是直接破坏原对象，而是在通信或执行路径中插入一个可控节点，从而实现观察、修改、转发或拦截。

3. 工程实现中的两个典型问题

在真正编写钩子库时，最常见的问题主要有两个：一是线程重入问题，二是指令截断问题。线程重入会导致多个线程同时进入被钩取函数，从而引发执行流混乱甚至程序崩溃；指令截断则会破坏原始指令边界，使恢复代码无法正确执行。这说明工程化 Hook 远比实验中的手工修改更复杂，需要考虑线程安全、指令长度分析和地址修正等问题。

表 1: 双重跳转技术在不同场景中的作用对比

应用场景	作用方式	主要目的
恶意代码 / 病毒	劫持入口、插入隐藏逻辑、再跳回	感染、隐藏、控制
安全软件 / 反病毒	拦截关键函数、检查后恢复执行	监控、防护、审计
外挂 / 反外挂	接管关键逻辑或检测入口篡改	对抗、检测、限制
调试 / Hook 框架	插入调试逻辑、记录后再返回	分析、测试、跟踪

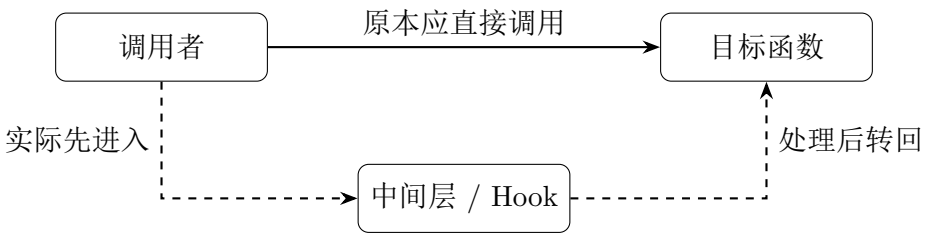

图 18 代码钩子与“中间层”思想示意图

七、总结及心得体会：

通过本次“病毒初见”实验，我对恶意代码模型的执行机制有了更加直观的认识。以前对这类内容的理解更多停留在概念层面，例如“修改程序入口”“劫持执行流”“恢复原始指令”等，而在本次实验中，我真正通过反汇编分析、机器码计算、内存修改和文件修改，把这些概念落实到了具体操作之中。特别是在 `normalfunc()` 入口写入首跳指令、在 `virusfunc()` 空闲区域恢复原始函数序言并构造回跳逻辑的过程中，我更加清楚地理解了程序控制流是如何被改变，又是如何被恢复的。

本实验中让我印象最深的是对 `JMP` 指令偏移量的计算和对完整指令边界的把握。刚开始时，会觉得只要写入一条跳转指令即可完成感染，但在实际操作中发现，真正困难的并不是“跳过去”，而是如何在跳走之后保证程序还能“跳回来”并继续正常运行。这就要求不仅要正确定位源地址和目的地址，计算出首跳和回跳的机器码，还必须确保被覆盖的原始指令能够在新的位置被完整恢复。也正因为如此，我进一步认识到机器码修改并不是简单地替换几个字节，而是需要结合函数结构、指令长度和控制流进行整体分析。

此外，本实验也让我体会到了不同工具在二进制分析中的作用差异。Visual Studio 更适合结合源码与反汇编观察程序结构，便于理解程序原始逻辑；ImHex、UE、C32ASM 等文件编辑工具更适合直接从文件层面观察和修改字节内容；x64dbg 调试套件中的 `x32dbg` 则兼具反汇编分析、汇编编辑和落盘保存等能力，更适合作为调试与修改的综合平台。通过分别在内存、硬盘文件和调试器三个层面完成相同模型的重现，我更加深刻地理解了三者在本质上的一致性，即它们最终都是围绕“修改机器码、改变控制流、恢复原逻辑”这一核心思想展开的。

总体来看，本实验不仅让我理解了恶意代码模型的基本原理，也锻炼了我在地址定位、反汇编阅读、机器码分析和调试操作等方面的能力。通过本次实验，我对后续进一步学习 Hook 技术、二进制分析、逆向工程和系统安全相关内容也有了更清晰的认识和兴趣。

八、对本实验过程及方法、手段的改进建议：

本实验内容具有较强的实践性，能够帮助学生从底层角度理解程序控制流劫持与恢复

的过程，但从完成实验的实际体验来看，仍有一些方面可以进一步优化，以便提高实验效率和教学效果。

首先，建议在实验指导资料中增加更细致的地址计算示例，尤其是对首跳偏移量、回跳偏移量、小端存储方式以及“为什么 JMP 要用源地址加 5 作为基准地址”等关键问题进行分步说明。虽然课堂讲解中已经涉及这些内容，但在实际操作时，学生往往仍会在公式理解和字节转换上出现混乱。如果能在实验手册中给出一组完整样例，学生在独立操作时会更加顺利。

其次，建议对“按完整指令覆盖”的问题作更明确强调。本实验中一个很重要的点，就是不能只机械地覆盖固定字节数，而必须分析被覆盖区域对应的是哪些完整指令，并在新的位置恢复这些指令。对于初学者而言，这一点往往比 JMP 偏移量计算更容易被忽略。若在实验说明中专门增加“常见错误分析”部分，例如指令截断、回跳位置错误、恢复代码不完整等情形，将有助于学生更好地理解实验中的关键细节。

再次，建议在实验前增加一个简短的工具预训练环节。例如，先让学生单独练习在 Visual Studio 中打开反汇编窗口、在内存窗口中修改字节、在十六进制编辑器中搜索特征码、在 x32dbg 中使用汇编编辑功能等。这样可以减少学生在正式实验过程中因工具不熟悉而花费过多时间，提高实验的整体完成效率。

报告评分：

指导教师签字：